WalletPlatform — Aurelian

A production-grade digital wallet and loyalty platform

Capgemini

A Training Report

Submitted in partial fulfilment of the requirements for the award of degree of

B.Tech in Information Technology (IT)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB

From 18/12/2025 to 20/05/2026

SUBMITTED BY:

Name of Student: Anuska Palit
Registration Number: 12206089
SuperSet ID: 6883801

STUDENT DECLARATION

To whomsoever it may concern

I, Anuska Palit(12206089), hereby declare that the work carried out by me on the project titled **"WalletPlatform – Microservices Wallet & Payment Platform"** during my industrial training at Capgemini is an original piece of work completed as a partial fulfilment of the requirements for the award of the degree of Btech of Computer Applications/ Information Technology.

The project work, analysis, design, implementation, and documentation presented in this report are based on the knowledge and skills acquired during the training period. The contents of this report have not been submitted previously, either in part or in full, for the award of any degree, diploma, or certification from any institution.

I further declare that all references, resources, and documentation consulted during the development of this project have been appropriately acknowledged.

Anuska Palit Date: 4/06/2026

Signature of Student

ACKNOWLEDGEMENT

I'm grateful to Lovely Professional University for introducing me to Industrial Training at Capgemini as a part of the BCA curriculum. The training was a great way to connect academics with the real world and provided me with first-hand exposure to software development in industry.

My gratitude also goes to Capgemini for putting together a thorough and industry-specific training program to learn about .NET Full Stack Development, with exposure to the following modern day technologies: C#, ASP.NET Core Web API, Entity Framework Core, SQL Server, Angular, RabbitMQ, Microservices Architecture, API Gateway Patterns, Authentication and Authorization Mechanisms and Enterprise Application Development.

I am also grateful to the trainers, mentors, evaluators and subject matter experts whose invaluable guidance, constructive recognition and insight helped me develop a comprehensive understanding of software architecture, distributed systems, secure application development and engineering best practices as applied in modern business organizations.

Finally, a special thanks to the project reviewers and assessment teams for their valuable input during the project evaluation phases to improve both quality and technical depth of my project.

These learnings and practical insights were put into good use in creating my capstone project titled “WalletPlatform – Microservices Wallet & Payment Platform” where I built a complete digital wallet system using microservices and event-driven architectures.

Lastly, I would like to express my gratitude to my family, friends, and well-wishers who provided me with constant encouragement throughout my learning journey.

Anuska Palit

Bachelor of Technology (Information Technology)

Lovely Professional University

Table of Contents

Introduction to the Training

Industrial training has a significant role in closing the gap between the skills needed by the industries today and the education provided by the colleges. The training gives the students an opportunity of getting a better understanding of the real-world software development methodologies, current technologies, enterprise processes, and professional engineering standards.My training at Capgemini which was part of my Bachelor of Technology (Information Technology) program from Lovely Professional University began on December 18, 2025 and ended on May 20, 2026.The training program was built around .NET Full Stack Development and was designed to provide a deep and thorough understanding of creating enterprise applications that utilize contemporary technologies and architectural patterns.

The course was organized in a systematic approach to learning from the fundamentals of program construction to the advanced technologies and techniques of microservices architecture, event-driven systems, API gateways, secure authentication, and front end application construction. Throughout the training program, I was able to have hands-on experience in developing and implementing scalable and maintainable software solutions using C#, ASP.NET Core, Entity Framework Core, Microsoft SQL Server, Angular Framework, RabbitMQ, and Ocelot API Gateway. This training program focused on not just understanding the concepts of developing enterprise applications but also on how to implement the learned concepts with a balance of theory and practical application through assignments, assessments, coding exercises, and project-based learning.The training was broken down into several phases with different areas of full stack software development covered. At first, the emphasis was on the C# language, and object-oriented programming principles. Following that, there were modules related to building web applications using ASP.NET Core MVC framework, and creating RESTful Web APIs. Later came more advanced concepts including database management, authentication and authorization, microservices, messaging using RabbitMQ, and frontend development using Angular.

Another important part of the training included a capstone project named “WalletPlatform – Microservices Wallet & Payment Platform”. In the project, I got a chance to utilize the knowledge I acquired during the whole course to build a realworld product. The application was built using the microservices architecture and had authentication, KYC verification, wallet management, payments, rewards management, notifications, and administrative capabilities.

There was also a lot of emphasis on the best practices of software engineering including clean architecture, layered architecture, API documentation, testing strategies, versioning using Git, secure coding, and so forth.

In summary, the industrial training I received at Capgemini has been very useful to expose me to the industry as well as improve my technical skills and problem-solving capabilities. This has equipped me more professionally for careers in software engineering and full stack application development.

Technical Learning from the Training

Through the .NET Full Stack Development Training in Capgemini, extensive experience was gained in relation to software development technologies. This training was an eye-opener in that it covered both backend and frontend aspects of software development.

i. **C# and Object-Oriented Programming**

The course started with basic concepts of C# programming language and then gradually moved on to advanced topics. Some topics that were discussed in depth include data types, control statements, collections, exception handling, delegates, events, generics, LINQ, asynchronous programming, and object-oriented programming principles. Particular focus was laid on encapsulation, inheritance, polymorphism, and abstractio.

ii. **ASP.NET Core and Web API Development**

This training course included substantial training in the ASP.NET Core framework, which is one of the most popular frameworks used for developing enterprise-level applications. Routing, middleware, dependency injection, model validation, configuration management, and the development of RESTful APIs were some of the topics discussed during the training.

The concept of REST architecture was understood thoroughly, especially by creating an API capable of performing CRUD operations, proper status code management, validation of requests, and consistent responses.

iii. **Entity Framework Core and SQL Server**

Management of data and persistence was done using Entity Framework Core and SQL Server. This training covered various aspects like Code-First approach, database migrations, relationships between entities, indexing, query optimization, and repository pattern.

iv. **Authentication and Authorization**

Security was one of the essential elements incorporated into the course of study. The training considered a range of authentication and authorization methods, particularly JSON Web Tokens (JWT)-based authentication. The subject areas included access token issuance, refresh token approaches, role-based authorization, password encryption with BCrypt, and securing APIs.Later, these aspects were successfully utilized in the development of WalletPlatform.

v. **Microservices Architecture**

Modern approaches to microservices architecture were examined during the training session. Among the topics discussed, there were service decomposition, bounded contexts, database per service architecture, API Gateway design, and inter-service communications.

Particular attention was paid to implementing loosely-coupled services able to function independently. This approach became the basis for the development of the WalletPlatform.

vi. **RabbitMQ and Event-Driven Communication**

RabbitMQ was introduced to ensure communication between distributed services. During the training, the topics covered include publishers, consumers, exchanges, queues, routing keys, and asynchronous messaging design.

This approach allowed avoiding tight coupling between different services.

vii. Angular Frontend Development

The training involved learning Angular and TypeScript skills for frontend development. Components, services, dependency injection, routing, reactive forms, HTTP client, route guard, and state management topics were covered in the training.Angular made possible the creation of interactive and responsive user interfaces that could interact effectively with backend microservices via REST APIs..

viii. API Gateway and Distributed Systems

In addition, I learned how to use the Ocelot API Gateway to route traffic from the client to the backend microservices. Gateways make it easier to communicate with clients, centralized authentication, rate limit, and act as a point of entry to a distributed system.

Through this, enterprise-level distributed systems could be implemented practically.

ix. Version Control and Development Practices

For source code management and collaboration during the training, Git was applied. Repositories, commits, branching, merging, pull request, and versioning were part of what I learned.

Further, software engineering best practices such as layering, clean coding, code review, documentation, testing, and debugging were taught in detail.

x. Learning Outcomes

By the end of the training, I had acquired practical experience in developing secure, scalable, and maintainable full-stack applications using modern technologies. The knowledge gained in backend development, frontend engineering, database design, distributed systems, and software architecture enabled me to successfully design and implement the WalletPlatform project as a production-grade microservices-based application.

Project Details – WalletPlatform

Project Overview

WalletPlatform is an industry-standard digital wallet and payment system, which was implemented in the final project of the Capgemini .NET Full Stack Development training program. WalletPlatform was created with the intention of replicating digital payment applications like Paytm, PhonePe, and Google Pay and implementing modern software engineering techniques.

WalletPlatform provides functionality for managing digital wallets, transferring funds between users' wallets, carrying out the KYC process, collecting loyalty rewards, using collected points, tracking transaction history and much more, all within one app. WalletPlatform is implemented based on the microservices architectural approach.

This training program project was used to practice all learned technologies and concepts, such as ASP.NET Core Web APIs, Angular, SQL Server, RabbitMQ, JWT Authentication, Ocelot API Gateway, Entity Framework Core and event-based communication.

Problem Statement

Conventional monolithic payment systems often suffer from issues concerning scalability, manageability, deployment complexity, and failure isolation. When users' activities grow, it becomes increasingly hard to scale up the system and avoid performance bottlenecks.

The goal of developing the WalletPlatform was to create a modern digital payment ecosystem that would handle all the necessary services, such as wallet management, payments, rewards, notification processing, and administration functions, by means of the microservices approach.

Objectives of the Project

The primary objectives of WalletPlatform were:

- To develop a secure digital wallet platform using microservices architecture.
- To implement user authentication and authorization using JWT-based security.
- To provide KYC verification workflows for user identity validation.
- To enable wallet funding, money transfers, and transaction management.
- To maintain financial integrity through ledger-based accounting mechanisms.
- To implement a rewards and loyalty programme for user engagement.
- To establish asynchronous communication using RabbitMQ.
- To create a scalable architecture that supports independent service deployment.

- To provide administrative controls for monitoring and managing the platform.

User Roles

The platform supports multiple user roles to ensure proper access control and business operations.

- **User:** Users can register on the platform, complete KYC verification, manage their wallets, transfer funds, view transaction history, earn rewards, redeem loyalty points, and receive notifications related to their activities.
- **Admin:** Administrators are responsible for reviewing KYC submissions, approving or rejecting applications, freezing or unfreezing wallets, managing reward rules, monitoring transactions, and overseeing platform operations.
- **Agent:** Agents act as operational users who assist with customer onboarding, verification processes, and support-related activities within the platform.

Key Features

WalletPlatform provides a comprehensive set of features designed for modern digital payment systems:

- Secure user registration and login
- JWT-based authentication and authorization
- Refresh token mechanism for session management
- KYC submission and approval workflow
- Wallet funding and withdrawal management
- Transaction history and audit tracking
- Rewards and loyalty points system
- Catalog-based reward redemption
- Voucher generation and redemption
- Multi-channel notification system
- Administrative dashboard and controls

Microservices Overview

The application follows a distributed microservices architecture where each service has its own responsibility and database.

1. **Auth Service**

   Handles user registration, login, authentication, authorization, JWT generation, refresh tokens, and user identity management.

2. **User Profile Service**
   Manages user profile information, KYC records, personal details, and profile updates.
3. **Wallet Service**
   Responsible for wallet creation, balance management, deposits, withdrawals, and wallet lifecycle operations.
4. **Ledger Service**
   Maintains immutable financial records and transaction history to ensure accounting accuracy and auditability.
5. **Rewards Service**
   Calculates reward points, manages loyalty tiers, and processes reward-related activities.
6. **Catalog Service**
   Maintains redeemable products, vouchers, and reward redemption operations.
7. **Notification Service**
   Delivers notifications through email and SMS channels for various platform activities.
8. **Receipts Service**
   Generates transaction receipts and maintains payment acknowledgement records.
9. **Admin Service**
   Provides administrative functionalities including KYC review, wallet management, monitoring, and operational controls.

Business Value

WalletPlatform demonstrates how modern financial technology solutions can be developed using enterprise-grade architectural patterns. The project highlights the practical implementation of distributed systems, secure payment processing, asynchronous communication, and scalable software design.

Beyond fulfilling the requirements of the training programme, the project serves as a complete demonstration of full-stack development capabilities and reflects industry-standard practices used in large-scale fintech applications.

System Architecture and Workflow

System Architecture

The WalletPlatform uses the Microservices Architecture along with Event-Driven Communication concepts. Instead of developing an application as a monolith, the system is split into many small independent services each performing a certain business logic operation. In this way, the performance, scalability, maintainability, and deployability of the platform improve significantly.

All the requests received from clients are redirected to the Ocelot API Gateway, which is used as the only entry point in the system. It performs all the operations related to the routing of requests, validation of authentication/authorization credentials, and communication with backend services. In such a way, each service remains independent while being an integral part of the platform.

Synchronous and asynchronous communication channels are implemented in the system to provide communication between backend services. REST APIs are utilized for real-time interactions, and RabbitMQ is applied for asynchronous event-based communication.The architecture consists of the following major components:

- Angular Frontend Application
- Ocelot API Gateway
- Authentication Service
- User Profile Service
- Wallet Service
- Ledger Service
- Rewards Service
- Catalog Service
- Notification Service
- Receipts Service
- Admin Service
- SQL Server Databases
- RabbitMQ Message Broker

This distributed design enables each service to be independently developed, tested, deployed, and scaled according to business requirements.

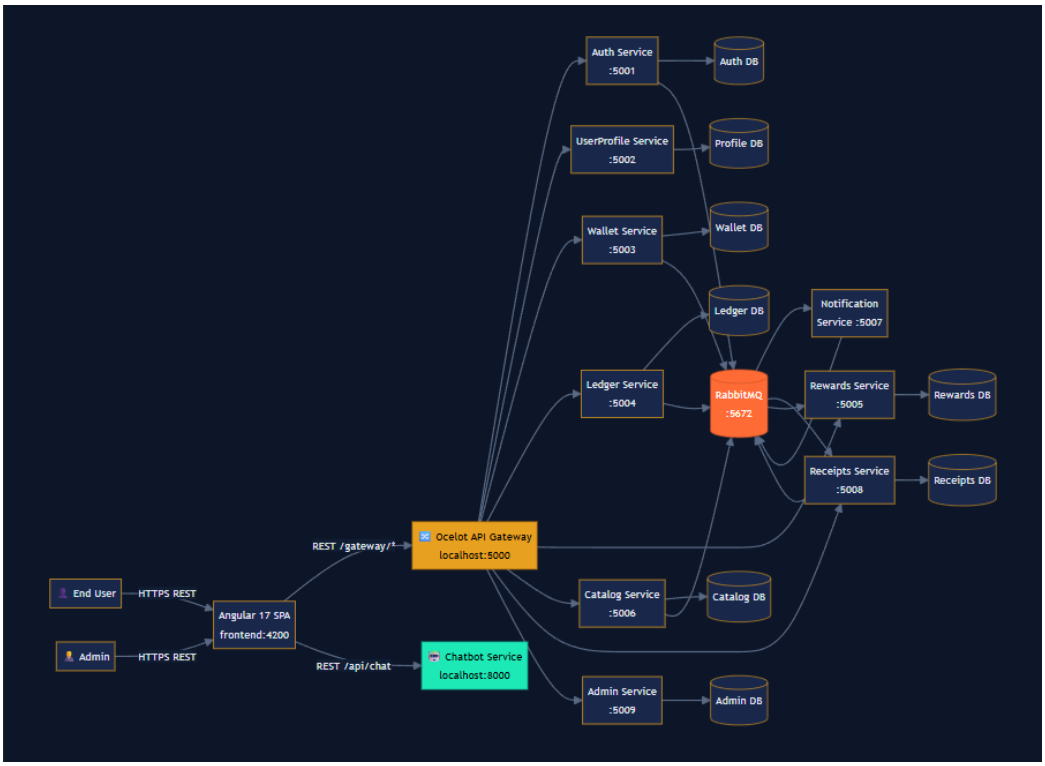


Figure I: System Architecture

API Gateway Workflow

The Ocelot API Gateway acts as the centralized access point for all external requests. When a user interacts with the application, requests are first sent to the gateway. The gateway validates authentication tokens, applies routing rules, and forwards requests to the appropriate microservice.

The use of an API Gateway offers several advantages:

- Centralized authentication and authorization
- Simplified client communication
- Request routing and load distribution
- Enhanced security and monitoring
- Reduced client-service coupling

This approach prevents direct exposure of internal services and improves overall system security.

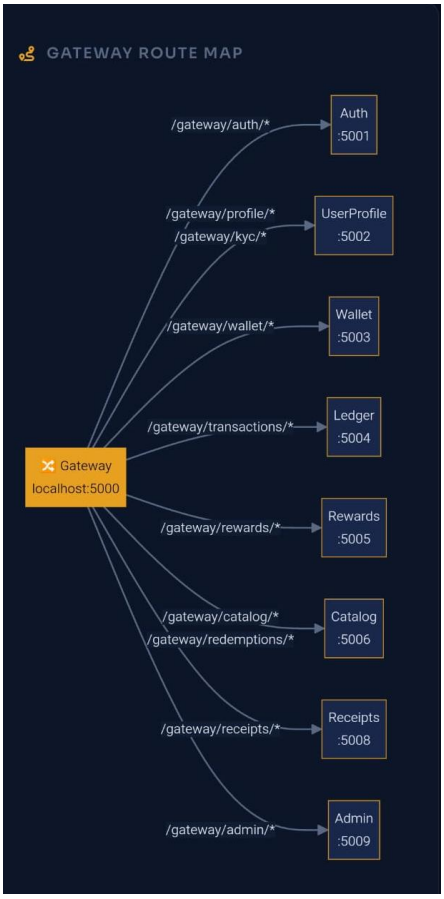


Figure ii: Gateway routings

Event-Driven Communication using RabbitMQ

To minimize direct dependencies between services, WalletPlatform implements event-driven communication through RabbitMQ. Services publish events whenever significant business actions occur, and other services subscribe to those events based on their responsibilities.

Examples of event-driven interactions include:

- User registration events
- Wallet creation events
- KYC approval events
- Payment completion events
- Reward allocation events
- Notification generation events

This asynchronous communication model improves system resilience and allows services to continue functioning independently even when other services are temporarily unavailable.

Queue Reference

QUEUE NAME	TRIGGER EVENT	CONSUMER	ACTION
wallet.creation.queue	UserRegistered	Wallet Service	Create wallet for new user
rewards.user.registered.queue	UserRegistered	Rewards Service	Seed rewards account + welcome bonus
user.registered.notification.queue	UserRegistered	Notification Service	Send welcome email
transaction.completed.wallet.queue	TransactionCompleted	Wallet Service	Adjust wallet balances
transaction.completed.rewards.queue	TransactionCompleted	Rewards Service	Award loyalty points
transaction.completed.receipts.queue	TransactionCompleted	Receipts Service	Generate transaction receipt
transaction.completed.notification.queue	TransactionCompleted	Notification Service	Email notification to parties
transaction.failed.notification.queue	TransactionFailed	Notification Service	Failure alert email
wallet.frozen.notification.queue	WalletFrozen	Notification Service	Freeze alert to user
kyc.updated.notification.queue	KYCStatusUpdated	Notification Service	KYC decision email
points.redeemed.queue	PointsRedeemed	Future consumers	Analytics / reporting
dead.letter.queue	Any failed msg	Operations	Dead-letter inspection

Figure iii: RabbitMQ reference table

User Registration Workflow

The user registration process begins when a new user submits registration details through the application interface. The Authentication Service validates the information, checks for duplicate accounts, securely hashes the password, and stores the user record.

Once registration is successful, a registration event is published through RabbitMQ. Other services consume this event and perform their respective operations such as profile initialization, wallet creation, and notification generation.

This workflow ensures loose coupling between services while maintaining business consistency across the platform.

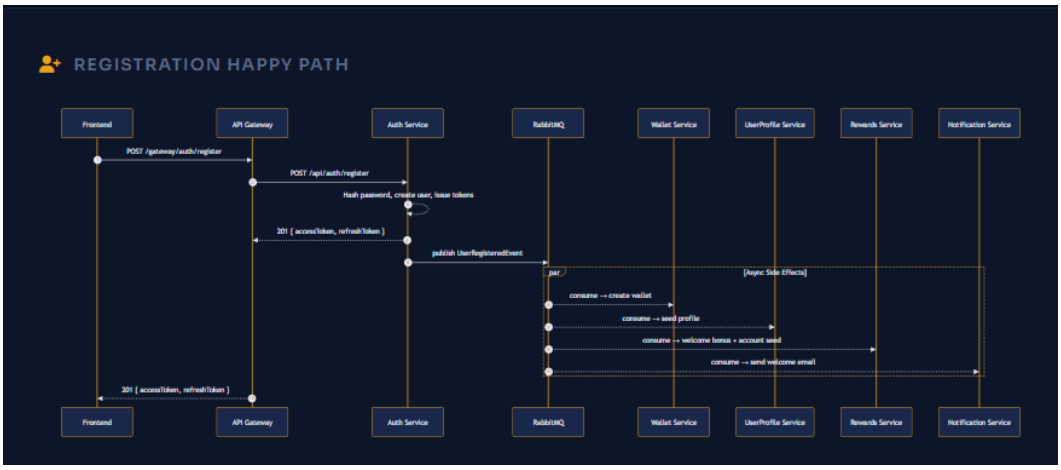


Figure iv: New user registration flow

KYC Verification Workflow

KYC (Know Your Customer) verification is implemented to validate user identity before enabling full wallet functionality.

The workflow consists of the following steps:

1. User submits identity documents.
2. KYC information is stored in the system.
3. Admin reviews submitted documents.
4. Documents are either approved or rejected.
5. Upon approval, the user's wallet becomes active.
6. Notification events are generated and delivered to the user.

This process ensures regulatory compliance and prevents unauthorized usage of financial services.

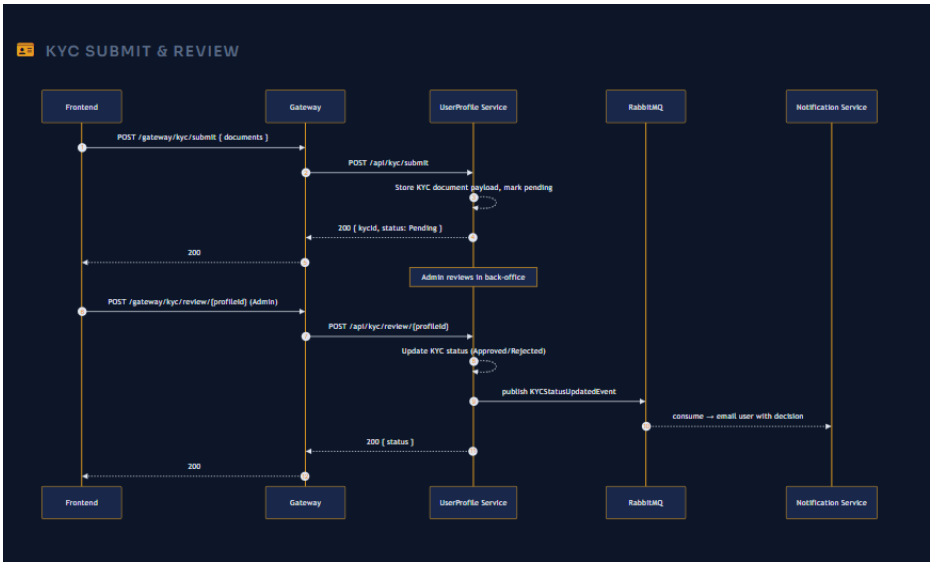


Figure v: Kyu submit and review flow

Payment Transaction Workflow

The payment transaction workflow represents one of the most critical components of the platform.

When a user initiates a transfer, the Wallet Service validates the transaction request and verifies available balance. After validation, the Ledger Service records immutable financial entries to maintain accounting accuracy.

The transaction process follows principles such as:

- Atomic transaction execution
- Double-entry accounting
- Idempotent payment processing
- Audit trail generation
- Failure recovery mechanisms

Once the transaction is completed successfully, events are published to trigger reward allocation, receipt generation, and user notifications.

This workflow ensures data consistency, financial integrity, and complete traceability of every transaction performed within the platform.

Benefits of the Architecture

The architecture adopted in WalletPlatform provides several advantages:

- Independent deployment of services
- Improved fault isolation

- Better scalability and maintainability
- Enhanced security through centralized access control
- Faster development and testing cycles
- Support for future expansion and integrations
- Efficient handling of high transaction volumes

The combination of microservices architecture, event-driven communication, and secure API management makes WalletPlatform a robust and scalable fintech solution capable of supporting modern digital payment operations.

Technology Stack and Database Design

Technology Stack

WalletPlatform was developed using a modern full-stack technology ecosystem capable of supporting scalable, secure, and maintainable enterprise applications. The selected technologies were chosen to ensure high performance, reliability, and flexibility while following industry best practices.

Backend Technologies

The backend services were developed using ASP.NET Core Web API on the .NET platform. C# served as the primary programming language for implementing business logic, service communication, authentication mechanisms, and transaction processing workflows.

Entity Framework Core was used as the Object Relational Mapper (ORM) to simplify database interactions and implement the Code-First development approach. This enabled efficient data management, schema migrations, and relationship handling.

Frontend Technologies

The user interface was developed using Angular and TypeScript. Angular provided a component-based architecture for building dynamic and responsive web applications, while TypeScript improved code quality through strong typing and object-oriented programming capabilities.

The frontend communicates with backend microservices through RESTful APIs exposed via the API Gateway.

Database Technology

SQL Server was used as the primary database management system. The platform follows the Database-per-Service pattern, where each microservice maintains its own independent database. This approach reduces coupling between services and allows independent scaling and deployment.

Messaging and Communication

RabbitMQ was implemented as the message broker to support asynchronous communication between services. Event-driven communication allows services to exchange information without creating direct dependencies, improving scalability and fault tolerance.

Security Technologies

Security was implemented using:

- JWT (JSON Web Tokens) for authentication
- Refresh Tokens for session management

- BCrypt for password hashing
- Role-Based Access Control (RBAC)
- Secure API access through Gateway validation

API Gateway

Ocelot API Gateway was used as the centralized entry point for all client requests. It provides routing, authentication validation, authorization, and simplified communication between clients and backend services.

Development Tools

The project utilized several development and collaboration tools including:

- Visual Studio 2022
- SQL Server Management Studio
- Git and GitHub
- Postman
- Swagger/OpenAPI
- RabbitMQ Management Console

Database Design

The database architecture of WalletPlatform was designed according to microservices principles, where each service owns and manages its own data store. This ensures service independence and prevents direct database sharing between services.

Auth Database

The Authentication Database stores:

- User Accounts
- Login Credentials
- Roles and Permissions
- Refresh Tokens
- Authentication Records

This database is responsible for identity management and secure access control.

User Profile Database

The User Profile Database maintains:

- Personal Information
- User Profiles
- KYC Details
- Address Information
- Verification Records

It supports user onboarding and identity verification workflows.

Wallet Database

The Wallet Database stores:

- Wallet Information
- Current Balances
- Wallet Status
- Funding Records
- Transaction References

This database manages all wallet-related operations.

Ledger Database

The Ledger Database maintains immutable financial records for every transaction performed on the platform. Each transaction generates corresponding ledger entries to ensure financial accuracy and complete auditability.

Rewards Database

The Rewards Database stores:

- Loyalty Points
- Reward Rules
- Tier Information
- Redemption History

This enables implementation of the platform's loyalty programme.

Catalog Database

The Catalog Database manages:

- Redeemable Products
- Voucher Information
- Inventory Details
- Redemption Transactions

Notification Database

The Notification Database maintains:

- Email Notifications
- SMS Notifications
- Delivery Status
- Communication Logs

Database Design Benefits

The database architecture provides several advantages:

- Service Independence
- Better Scalability
- Improved Security
- Simplified Maintenance
- Fault Isolation
- Faster Deployment Cycles
- Easier Future Expansion

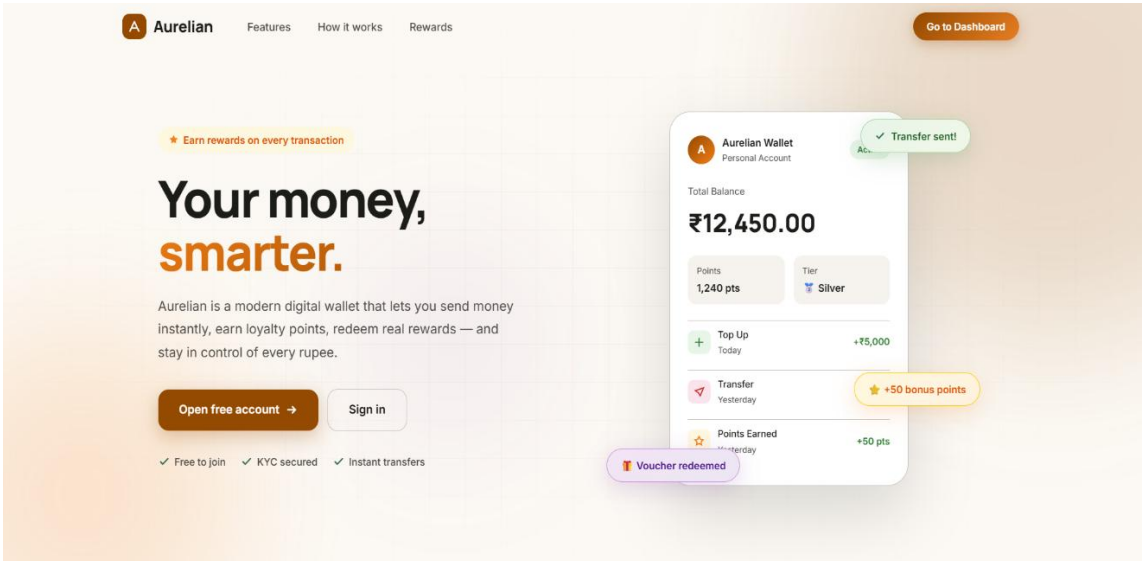
Figure vi: Database schema

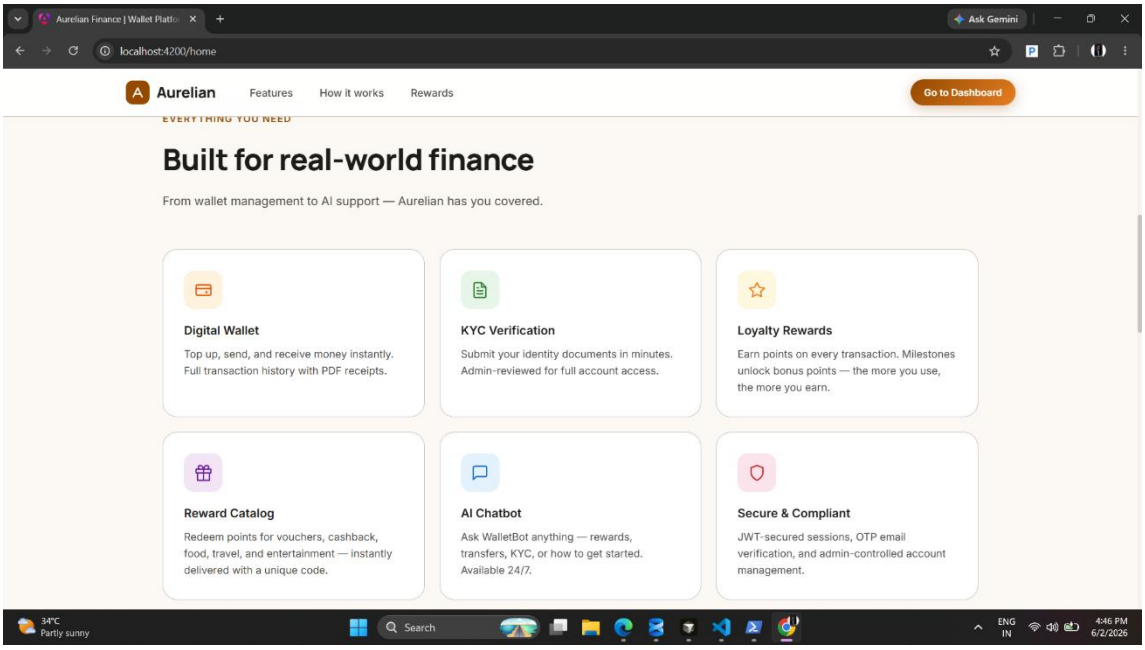
The combination of microservices architecture and database-per-service design enables WalletPlatform to handle increasing workloads while maintaining performance, reliability, and data consistency.

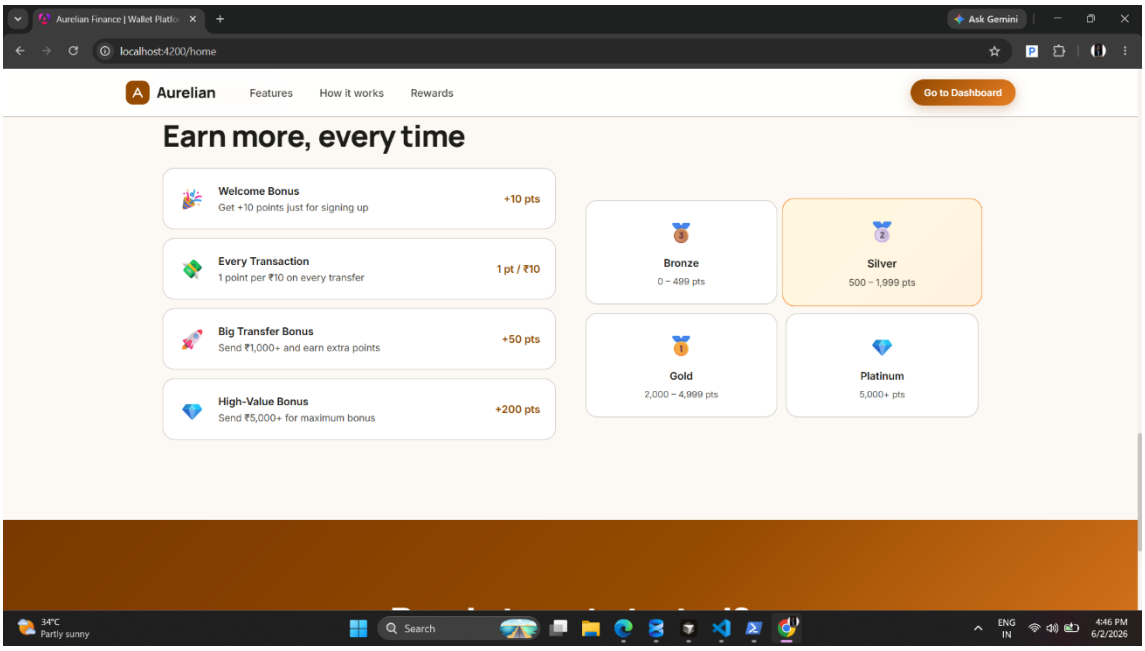
Interface Designs

Home Page

The Home Page serves as the public entry point of the WalletPlatform application. It introduces users to the platform's core services, including digital wallet management, KYC verification, money transfers, rewards, and AI-powered assistance. The interface is designed to provide a clear overview of the platform's features while encouraging user onboarding and engagement

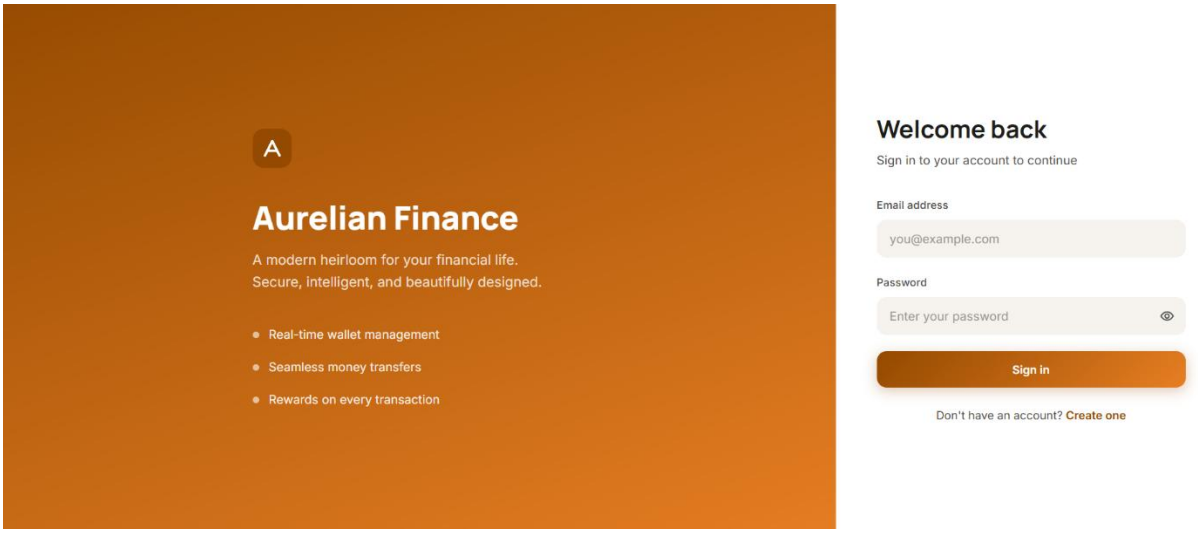






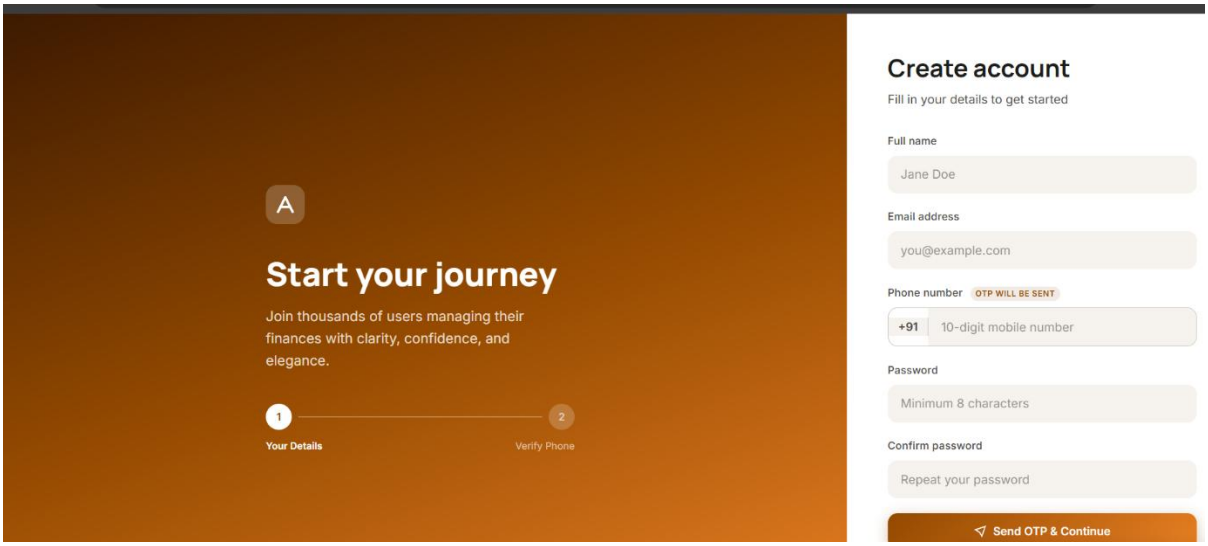
Login Page

The Login Page enables registered users to securely access their accounts using email and password credentials. The system utilizes JWT-based authentication to ensure secure session management and protected access to platform resources.



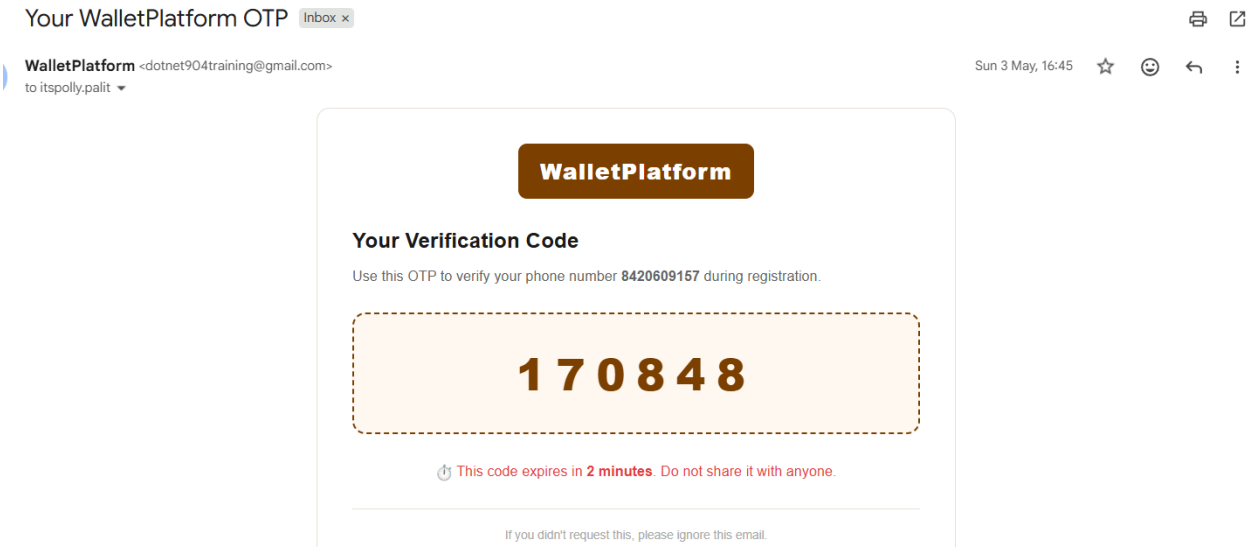
Registration Page

The Registration Page allows new users to create an account by providing personal information, contact details, and authentication credentials. OTP verification is incorporated to validate user identity and ensure secure account creation.



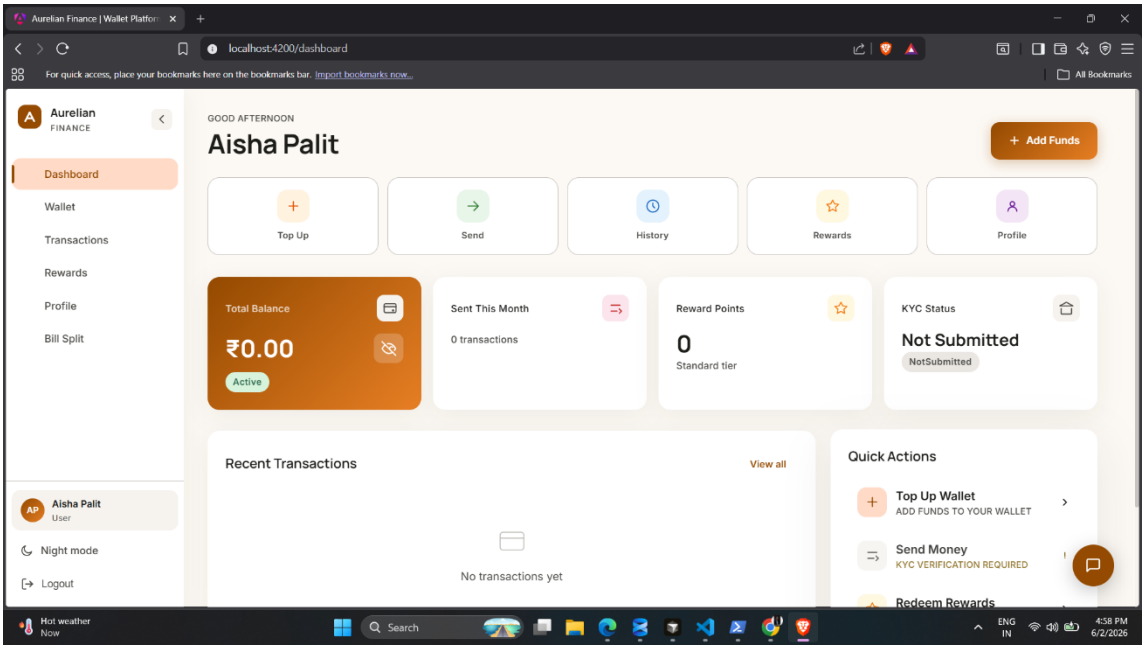
Email OTP Verification Page

The Email OTP Verification Page is used to validate the user's email address during the registration process. A One-Time Password (OTP) is sent to the registered email, and the user must enter the received code to complete verification. This additional security layer helps prevent unauthorized registrations and ensures the authenticity of user accounts.



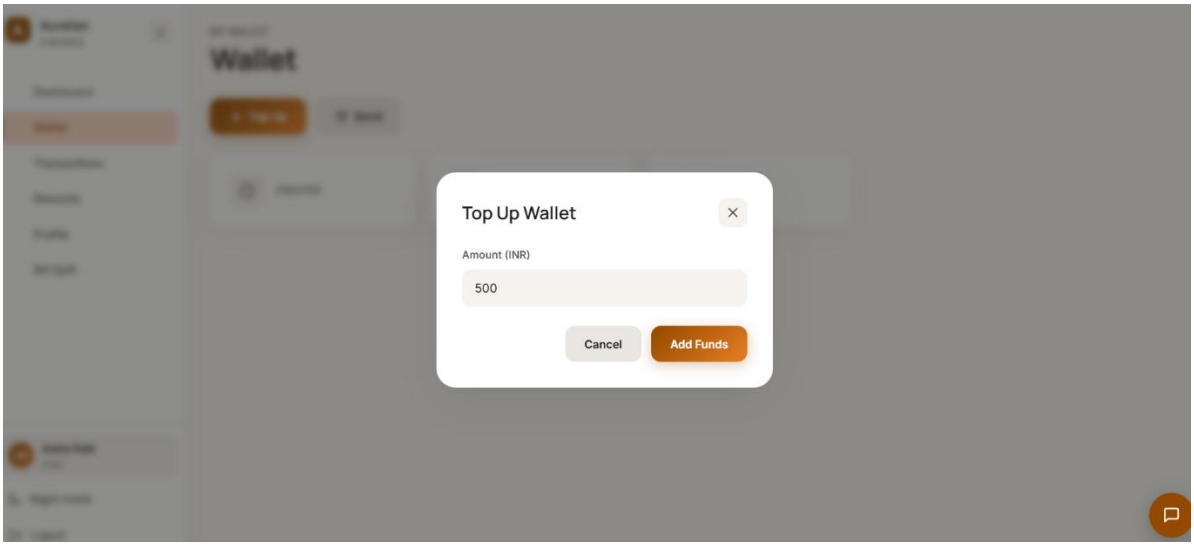
User Dashboard

The Dashboard provides users with a centralized view of their wallet information, recent transactions, reward points, and KYC status. It also offers quick access to frequently used operations such as wallet funding, money transfers, rewards management, and profile updates.



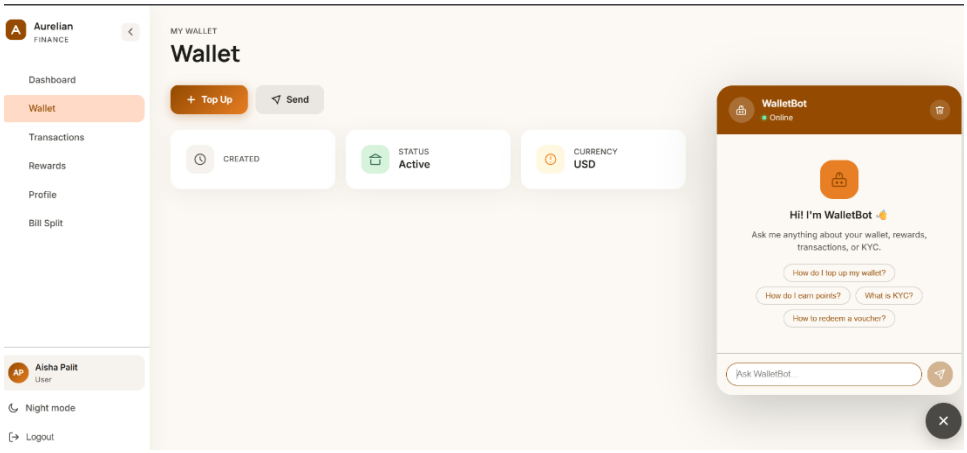
Top-Up Module

The Top-Up Module allows users to add funds to their digital wallet securely. Users can initiate wallet funding operations, monitor balance updates, and maintain sufficient funds for future transactions and platform activities.



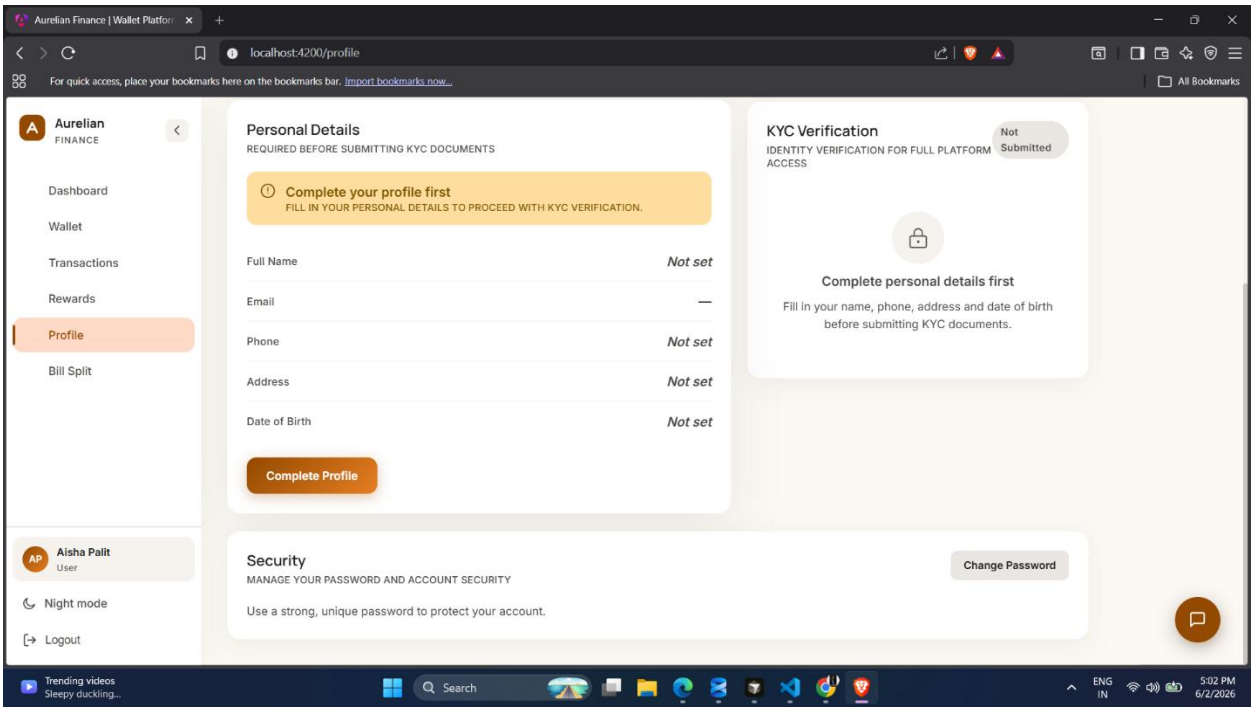
Wallet and AI Chatbot Module

The Wallet Module enables users to view wallet balances, transaction summaries, and account status. An integrated AI Chatbot assists users by answering queries related to wallet operations, rewards, KYC processes, and general platform navigation, thereby improving the overall user experience.



Profile Management Module

The Profile Management Module allows users to view and update personal information, manage account settings, and monitor verification status. This module ensures that user records remain accurate and up-to-date while supporting secure account administration.



Examinations and Evaluations

Throughout the training period, multiple assessments and evaluation sessions were conducted to measure technical understanding, practical implementation skills, and overall progress. These evaluations played a significant role in ensuring continuous learning and effective application of the concepts covered during the training programme.

M1 Assessment

The M1 Assessment was conducted during the initial phase of the training programme. The assessment focused on evaluating the understanding of C# programming fundamentals, object-oriented programming concepts, ASP.NET Core basics, database concepts, and problem-solving abilities.

The examination helped identify strengths and areas requiring improvement while establishing a strong foundation for advanced topics covered in later phases of the training.

Project Evaluation 1

The first project evaluation focused on the initial implementation of WalletPlatform. During this stage, the core architecture, authentication system, database design, and primary microservices were demonstrated.

The evaluation emphasized:

- Project architecture design
- Service decomposition strategy
- Authentication workflow
- Database implementation
- API development standards

Constructive feedback received during this evaluation helped improve system design and implementation quality.

Project Evaluation 2

The second evaluation was conducted after the completion of major functionalities of the project. At this stage, multiple microservices were integrated successfully, and advanced features such as wallet operations, KYC workflows, event-driven communication, rewards management, and notifications were demonstrated.

The evaluation assessed:

- Functional completeness

- Microservices communication
- Security implementation
- Code quality
- Database architecture
- API documentation

The review process ensured that the project aligned with enterprise software development practices.

Final Assessment (L1)

The final assessment represented the culmination of the training programme. It involved a comprehensive review of the project, technical concepts, architectural decisions, and implementation methodologies.

The evaluation included:

- End-to-end project demonstration
- Technical discussion and Q&A
- Architecture review
- Database design analysis
- Security implementation review
- Software engineering best practices

The successful completion of the final assessment demonstrated proficiency in full-stack development and enterprise application design.

Overall Evaluation Outcome

The assessments conducted throughout the training journey provided valuable opportunities to validate technical skills and receive expert feedback. They contributed significantly to the successful development of WalletPlatform and enhanced my confidence in applying modern software engineering practices within real-world projects.

Conclusion

The industrial training at Capgemini from 18th December 2025 to 20th May 2026 has been a highly valuable and enriching experience. The training provided a strong foundation in modern software development technologies and exposed me to industry-standard practices followed in enterprise application development.

Throughout the programme, I gained practical knowledge of C#, ASP.NET Core, Entity Framework Core, SQL Server, Angular, RabbitMQ, Microservices Architecture, API Gateway implementation, authentication mechanisms, and distributed system design. The structured learning approach helped me understand both theoretical concepts and their real-world applications.

The knowledge acquired during the training was successfully applied in the development of the project "WalletPlatform – Microservices Wallet & Payment Platform". The project provided hands-on experience in designing and implementing a secure, scalable, and maintainable application using microservices and event-driven architecture principles. It also strengthened my understanding of software architecture, database design, API development, security implementation, and system integration.

One of the most significant outcomes of the training was the opportunity to work with enterprise-level technologies and architectural patterns commonly used in modern organizations. The project implementation enhanced my problem-solving abilities, analytical thinking, debugging skills, and software engineering practices.

The various assessments, project evaluations, coding exercises, and technical discussions conducted throughout the training period played a vital role in reinforcing learning and identifying areas for continuous improvement. These experiences helped me develop both technical competency and professional confidence.

Overall, the training has significantly contributed to my academic and professional growth. It has prepared me for future opportunities in software development and provided a deeper understanding of full-stack application development using modern technologies. The experience gained during this training will serve as a strong foundation for my future career in the field of Information Technology.

References

- Microsoft Documentation — ASP.NET Core. https://learn.microsoft.com/en-us/aspnet/core/
- Microsoft Documentation — Entity Framework Core. https://learn.microsoft.com/en-us/ef/core/
- Microsoft Documentation — C# Language Reference. https://learn.microsoft.com/en-us/dotnet/csharp/
- Microsoft Documentation — SQL Server Documentation. https://learn.microsoft.com/en-us/sql/
- RabbitMQ Documentation. https://www.rabbitmq.com/documentation/
- Ocelot API Gateway Documentation. https://ocelot.readthedocs.io/
- Angular Documentation. https://angular.dev/
- RxJS Documentation. https://rxjs.dev/
- TypeScript Documentation. https://www.typescriptlang.org/docs/
- JWT.io — JSON Web Tokens Introduction. https://jwt.io/introduction/
- BCrypt.Net — Secure Password Hashing. https://github.com/BcryptNet/bcrypt.net
- Swagger / OpenAPI Specification. https://swagger.io/specification/
- Git Documentation. https://git-scm.com/doc
- GitHub Documentation. https://docs.github.com/
- WalletPlatform High Level Design (HLD).
- WalletPlatform Low Level Design (LLD).
- WalletPlatform API Reference Documentation.
- Capgemini .NET Full Stack Development Training Materials.